Distributed Multi-Model Analytics for E-commerce Data

Technical report, AUCA Big Data Analytics final project · Nov 11, 2026 · @Ndanyuzwe

1. Introduction

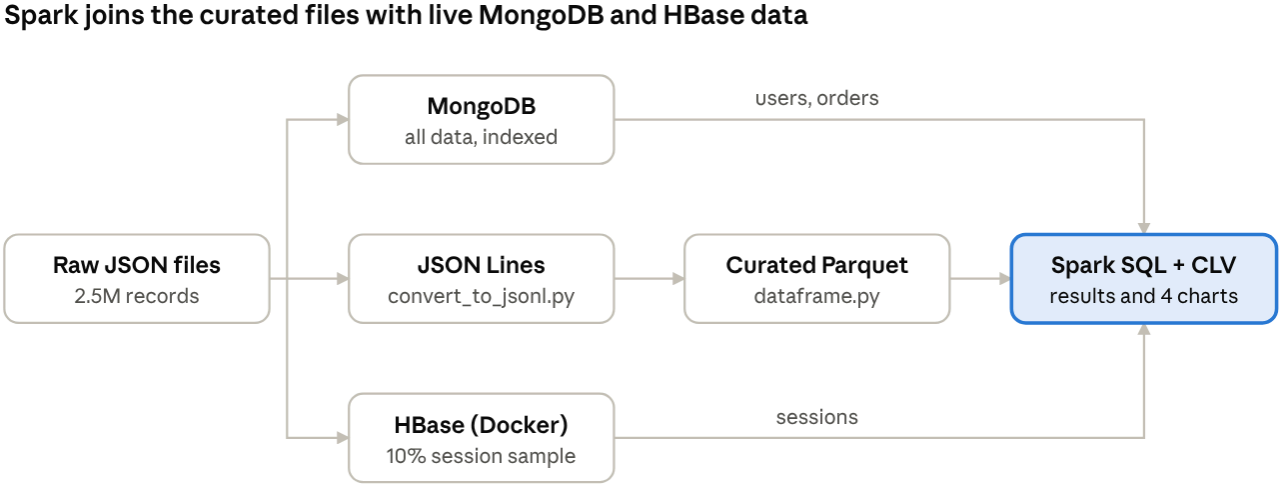
This project builds a working analytics system for an online store using three technologies: MongoDB (document store), HBase (wide-column store) and Apache Spark (distributed processing). It runs end to end on 2.5 million synthetic records, from raw JSON to cleaned tables, SQL analytics, a customer lifetime value (CLV) model and charts. All code and run instructions are in the repository (github.com/Duncanwar/Final-Project); the README lists every step.

The system answers four business questions:

1. Which products are popular, and do views turn into sales?
2. How do sales develop over time and across categories?
3. Which customers are most valuable, and what makes them valuable?
4. Which products are bought together, to drive recommendations?

The dataset, produced by the course's `dataset_generator.py`, covers 90 days: 25 categories, 5,000 products, 10,000 users, 500,000 transactions and 2,000,000 sessions.

2. System architecture and technology selection



Data flow · 3 stores, 7 stages

Technology	Used for	Why
MongoDB	All entities; aggregation pipelines	Nested data (order lines, page views) fits one document; flexible indexes and aggregations
HBase (Docker)	Session history per user; product metrics per day	Rows sorted by key give fast time-range scans at high write volume
Spark + Parquet	Cleaning, recommendations, SQL, joining the stores	Parallel processing of large files; SQL over any source loaded as a DataFrame

3. Data modeling

MongoDB document model

Data that is always read together is embedded; links between collections are references by id. Transactions embed their line items, sessions embed page views, device and location, products embed their price history, and categories embed their subcategories.

User profiles. A purchase summary was deliberately not embedded in the user document. With 500,000 orders arriving over time, an embedded summary would need an update on every order. Instead, MongoDB computes it on demand with one `$group` per customer (orders, revenue, first and last order), which takes seconds and is always current.

Validation and indexes. Each collection has a `$jsonSchema` validator for required fields, types and allowed values (for example `conversion_status`). Indexes follow the main queries: `user_id + timestamp` for a customer's order history, `user_id + start_time` for a user's sessions, multikey indexes on `items.product_id` and `viewed_products`, and `category_id` for the catalog.

HBase model

HBase stores rows sorted by row key, so each key is built around its read pattern:

- **`user_sessions`**, key `user_id#reverse_timestamp#session_id`, families `info` (session attributes) and `views` (one column per page view). One prefix scan returns a user's sessions, newest first.
- **`product_metrics`**, key `product_id#YYYYMMDD`, family `m` (views, view seconds, cart adds, quantity bought). One range scan returns a product's days.

The reverse timestamp (9,999,999,999 minus the start time) makes newer sessions sort first. Splitting the families lets analysis read only `info`. `hbase/hbase.py` creates the tables and runs these queries; in the HBase shell:

```
create 'user_sessions', 'info', 'views'
scan 'user_sessions', {ROWPREFIXFILTER => 'user_000123#', LIMIT => 5}
scan 'product_metrics', {STARTROW => 'prod_00042#20260901', STOPROW =>
'prod_00042#20261001'}
```

The user query returns that user's sessions in 0.7 seconds, the newest (27 October 2026) first. The `product_metrics` table holds 292,679 product-day rows built from the sample.

Why each data type is in MongoDB or HBase

MongoDB holds all data because it answers flexible questions on any field and joins inside one document. HBase holds the session history because it is a high-volume time series read by key range, which HBase does fastest, but it has no secondary indexes. Sessions are therefore kept in both stores, each serving the queries it suits.

4. MongoDB aggregation pipelines

`mongo_aggregations.py` defines two pipelines from the brief's list: product popularity and user segmentation by purchase frequency. Both run inside MongoDB, so only the top results leave the database.

Pipeline 1: most viewed products

`$unwind` splits each session's `page_views` array, `$match` keeps product detail views, `$group` counts views per `product_id`, and `$sort` with `$limit` keeps the top 10.

Unwinding about 20 million page views makes this the slow part of the script (several minutes on a laptop); Spark computes the same ranking faster (section 6, A4).

Pipeline 2: customers by purchase frequency

```
pipeline2 = [
    {"$match": {"status": "completed"}},
    {"$group": {"_id": "$user_id", "purchase_frequency": {"$sum": 1}}},
    {"$sort": {"purchase_frequency": -1}},
    {"$limit": 10},
]
db.transactions.aggregate(pipeline2)
```

`$match` runs first, so only completed orders are grouped. It runs in under a minute. The most frequent buyers (for example `user_009389`) completed 50 orders each in the 90 days.

5. Spark batch processing

`dataframe.py` reads the JSON Lines files and writes one cleaned Parquet table per entity. It trims and standardizes text, parses timestamps, casts numbers, flattens nested objects (device, location) and splits arrays into their own tables: 954,134 order lines and 109 subcategories. Row counts were checked at every step, and all 2.5 million records were kept.

"Customers who bought X also bought Y"

The job joins order lines to themselves on `transaction_id` to count how often two products are bought together. For each pair it computes support (share of all orders with both), confidence (share of orders with A that also have B) and lift (confidence divided by B's overall share; above 1 means more than chance).

Pairs bought together in fewer than 2 orders are dropped, and the top 5 partners by lift are kept for each product. The output has 18,990 recommendations covering 4,285 of the 5,000 products. For example, `prod_00283` is recommended with `prod_04494` at a lift of 18.1.

Most pairs share only 2 or 3 orders, because the data generator picks products mostly at random. The method is standard and would find stronger patterns in real purchase data.

Optimization choices

JSON arrays need Spark's `multiLine` option, which reads each file as one task and ran out of memory on the 8 GB laptop, so the files were converted to JSON Lines. Intermediate data is stored as Parquet, so later steps read only the columns they need. The sessions conversion is skipped when its `_SUCCESS` marker exists, making re-runs fast. Spark runs as `local[3]` with 3–4 GB of driver memory, and per-customer grouping is pushed down to MongoDB, so 10,000 rows reach Spark instead of 500,000 orders.

6. Spark SQL analytics

Part A: queries on the curated data

- **A1 Monthly revenue per category:** CTE, three-table join, `SUM() OVER` for share of month, `RANK()`, and `LAG()` for month-over-month growth.

- **A2 Daily revenue:** a 7-day moving average with a `ROWS BETWEEN 6 PRECEDING` window.
- **A3 Session funnel:** `GROUPING SETS` gives conversion by device, by referrer, by both and overall in one query. Conversion is about 20% everywhere (20.00% to 20.50%) across 2,000,000 sessions.
- **A4 Product performance:** `LATERAL VIEW explode()` on viewed products, joined to sales, with two `RANK()` windows.

Part B: queries on data loaded from MongoDB and HBase

`data_sources.py` loads 10,000 users and a `$sample` of 50,000 orders from MongoDB and 200,000 sessions from HBase into DataFrames (falling back to the Parquet copy if a store is down). **B1** joins them by the user's country: about 60 buyers per country, with LA top at $332,029 in sampled revenue. **B2** links each MongoDB order to its HBase session: tablet with email leads ($156,780 from 200 orders), mobile with social trails ($109,620 from 160).

7. Integrated analytics: customer lifetime value

Business question: how much is each customer likely to be worth over their lifetime, and does browsing activity explain that value?

Answer: the top quarter of buyers (Platinum) is worth $171,639 on average, 1.6 times the bottom quarter (Bronze, $110,475). Value is driven by order frequency, not browsing. `integrated_clv.py` computes this in one Spark SQL query over data from both stores.

Model: CLV = average order value × orders per month × profit margin × (1 ÷ monthly churn).

Monthly churn is the share of customers with no order in the last 30 days of the data window. Its inverse is the expected lifespan in months, capped at 36. Session counts from HBase are divided by the 10% sample fraction to estimate the full number. Buyers are then split into four equal segments with `NTILE(4)` on monthly value.

1. **MongoDB:** read the 10,000 user profiles; run a `$group` on transactions so MongoDB returns one summary row per customer; average the category profit margins.
2. **HBase:** scan `user_sessions`, reading only the `info` family, and split each row key back into user id and session id.
3. **Spark:** register the three DataFrames as SQL views, join them on `user_id`, score every customer and rank them into segments.

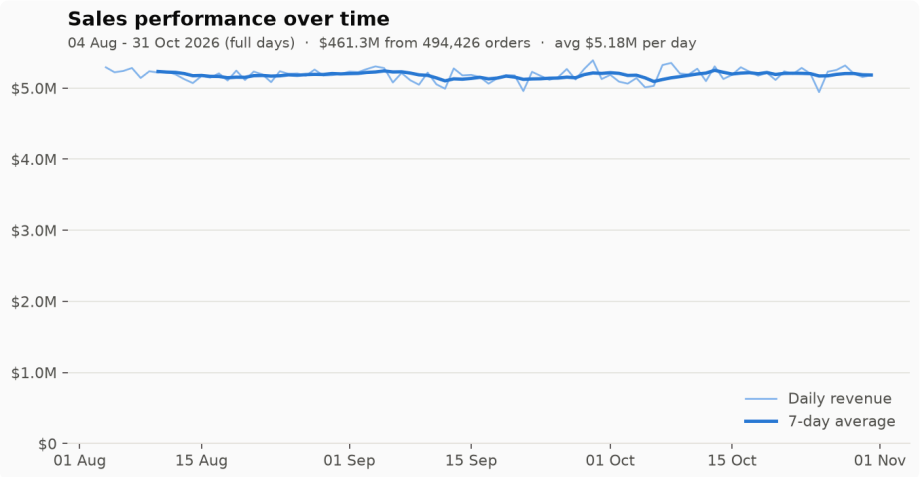
Segment (2,500 each)	Average CLV	Orders per month	Sessions per month	Average order value
Platinum	$171,639	19.1	68.1	$1,001
Gold	$147,603	17.3	66.8	$951
Silver	$131,814	16.0	66.1	$918
Bronze	$110,475	14.3	65.7	$863

Among buyers, CLV correlates strongly with orders per month (r = 0.82) but barely with sessions per month (r = 0.06), session conversion (r = 0.12) or session length (r = 0.00). Platinum customers visit only 4% more often than Bronze customers, but order 34% more often.

The measured churn was only 2.8% a month, so every customer reached the 36-month lifespan cap. The CLV figures therefore depend on that cap, and the ranking of customers is more reliable than the dollar amounts.

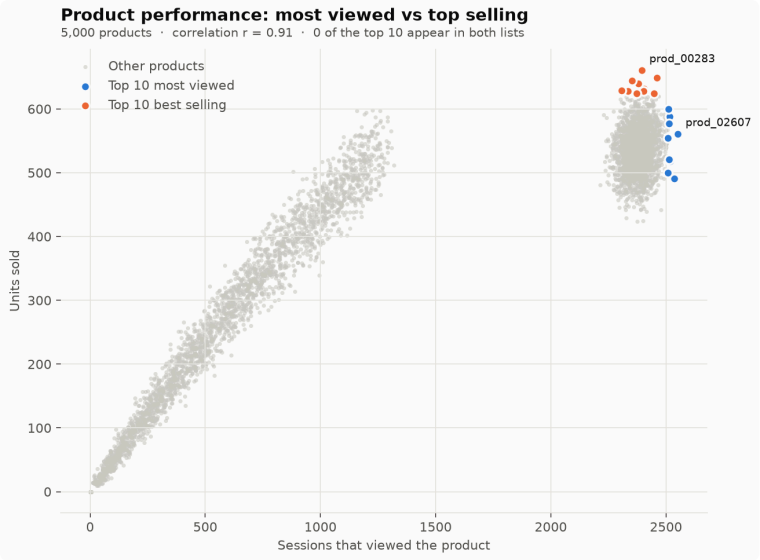
8. Visualizations and business insights

Figure 1: Sales performance over time



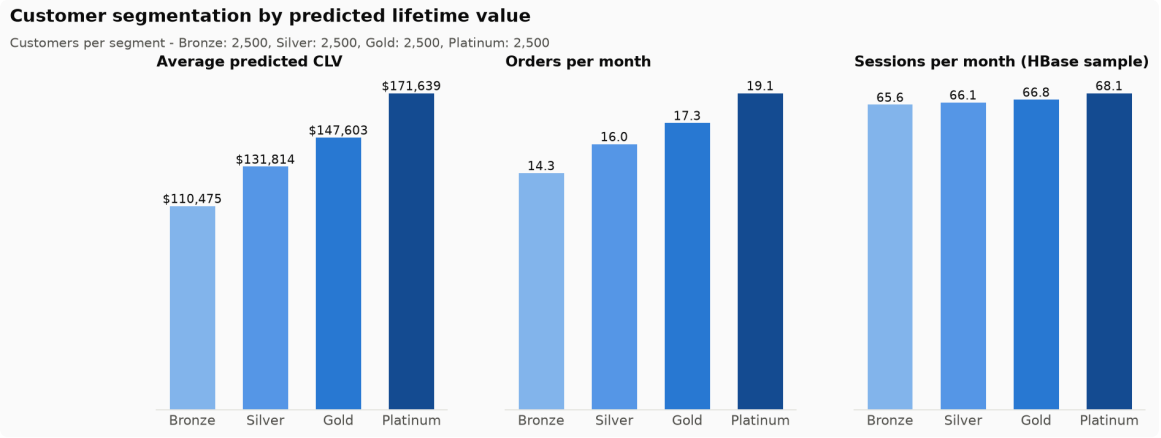
Revenue is stable at about $5.2 million a day across the 91-day window (3 August to 1 November 2026), with no trend or weekly cycle. The first and last days are cut off by the data window, so they are left out of the chart, and the 7-day average starts once a full week is available.

Figure 2: Most viewed vs top-selling products



Across 5,000 products, views and units sold are strongly correlated (r = 0.91), and products form two clusters. Even so, none of the 10 most viewed products is among the 10 best sellers. The most viewed product (`prod_02607`) ranks 540th in sales. Products with high views but low sales are candidates for price or page improvements.

Figure 3: Customer segments by lifetime value



The three panels compare the CLV segments on value, order frequency and browsing. Average CLV and orders per month both rise steadily from Bronze to Platinum, while sessions per month stay almost flat (65.7 to 68.1). This is the clearest visual evidence that order frequency, not browsing, drives customer value.

Recommended actions

1. **Invest in retention, not traffic.** Order frequency explains CLV (r = 0.82); browsing does not (r = 0.06). Loyalty offers and reorder reminders for Silver and Gold customers have the largest upside.

2. **Fix high-view, low-sale products.** Products such as `prod_02607` attract the most visits but rank in the hundreds for sales. Review their price, stock and product page.
3. **Use the "also bought" table** on product and cart pages, starting with the highest-lift pairs.
4. **Do not shift budget between channels on conversion alone.** All devices and referrers convert at about 20%.

9. Scalability considerations

- **MongoDB:** shard `sessions` and `transactions` on a hashed `user_id`, so each user's data stays on one shard; replica sets for availability.
- **HBase:** store the full data on HDFS across several RegionServers, with pre-split regions so new sessions do not all land on one server.
- **Spark:** run the same code on a cluster (YARN or Kubernetes) by changing the master URL; partition Parquet by date.
- **Bottleneck today:** the loaders in `data_sources.py` collect rows in Python before Spark sees them. That works for 200,000 rows; at larger scale the MongoDB Spark Connector and an HBase connector let each executor read its own part in parallel.

10. Limitations and future work

Limitations. The data is synthetic and evenly spread, so conversion and co-purchases vary little. HBase holds a 10% sample and everything runs on one machine. The CLV lifespan hit its 36-month cap, and with no age field, segments use CLV tiers.

Future work. Load all sessions into HBase and read both stores through Spark connectors; replace the CLV heuristic with a BG/NBD model; add Spark Structured Streaming over new session files to update `product_metrics` continuously.

11. Conclusion

The analysis gave three useful results. Sales are stable at about $5.2 million a day. Product views predict sales overall, but the most viewed products are not the best sellers. Customer value depends mostly on how often people order, which points to retention and repeat-purchase programs rather than more traffic.